

# Documentation Technique du Projet – Bibliothèque en Ligne

---

## 1. Présentation générale

### 1.1 Contexte et objectifs

Le projet Bibliothèque en Ligne est une application web développée dans le cadre du cours Développement Web – Niveau Intermédiaire. Il permet à des utilisateurs de consulter un catalogue de livres, d'effectuer des recherches, d'ajouter des ouvrages à une liste de lecture personnelle et de gérer la collection (CRUD complet) via une interface intuitive et sécurisée.

### 1.2 Fonctionnalités clés

- Gestion des livres : Ajout, modification, suppression, affichage des détails.
- Recherche : Par titre ou auteur avec résultats dynamiques.
- Liste de lecture (wishlist) : Chaque utilisateur peut ajouter/retirer des livres.
- Authentification : Inscription, connexion, déconnexion avec sessions.
- Upload d'images : Chaque livre peut avoir une couverture (JPEG, PNG, WEBP, GIF) avec renommage automatique.
- Responsive design : Adaptation aux écrans mobiles, tablettes et ordinateurs.

---

## 2. Architecture du projet

### 2.1 Structure des dossiers et fichiers

```
bibliotheque/
├── app/
│   ├── config.php      # Code métier (non accessible via navigateur)
│   ├── db.php          # Paramètres de connexion DB
│   ├── helpers.php     # Connexion PDO
│   ├── validations.php # Fonctions utilitaires (e(), sessions, flash)
│   ├── auth.php        # Validation des formulaires (livres)
│   ├── helpers_upload.php # Protection des pages (require_auth)
│   └── fonctions.php    # Upload et suppression d'images
├── public/
│   ├── index.php       # Requêtes métier (CRUD, wishlist, recherche)
│   ├── results.php     # Racine du site (accessible)
│   ├── details.php     # Page d'accueil + recherche + derniers livres
│   ├── wishlist.php    # Résultats de recherche
│   ├── add_book.php    # Détails d'un livre (ajout wishlist)
│   ├── edit_book.php   # Liste de lecture de l'utilisateur
│   ├── delete_book.php # Ajout d'un livre
│   ├── login.php       # Modification d'un livre
│   ├── logout.php      # Suppression (confirmation)
│   ├── register.php    # Connexion
│   └── assets/         # Déconnexion
│                       # Inscription
│                       # Ressources statiques
```

|                |                                                       |
|----------------|-------------------------------------------------------|
| css/           |                                                       |
|                |                                                       |
| └─ style.css   | # Feuille de style unique                             |
| js/            |                                                       |
|                |                                                       |
| └─ app.js      | # Validation client                                   |
| uploads/       | # Dossier des images uploadées (créé automatiquement) |
| sql/           | # Scripts SQL                                         |
|                |                                                       |
| └─ schema.sql  | # Structure de la base                                |
| └─ donnees.sql | # Données de test (compte admin + livres)             |
| README.md      | # Guide d'installation rapide                         |

## 2.2 Principe de séparation

- Le dossier `app/` contient tout le code réutilisable (connexion, helpers, requêtes). Il n'est pas directement accessible depuis le navigateur, ce qui renforce la sécurité.
- Le dossier `public/` est le point d'entrée unique (DocumentRoot). Seuls les fichiers nécessaires à l'affichage y sont placés.
- Les assets (CSS, JS, images uploadées) sont servis statiquement depuis `public/assets/`.

## 3. Installation et configuration

### 3.1 Prérequis

- Serveur local : Laragon, XAMPP, WAMP ou équivalent (Apache + MySQL + PHP ≥ 7.4)
- Extensions PHP : PDO, MySQLi, GD (pour l'upload), fileinfo (pour `mime_content_type`)
- Navigateur : Firefox, Chrome ou Edge récent.

### 3.2 Procédure d'installation

1. Télécharger ou cloner le projet dans le répertoire racine du serveur (ex : `C:\laragon\www\bibliotheque\`).
2. Créer la base de données :
  - o Ouvrir phpMyAdmin ou exécuter le script `sql/schema.sql`.
  - o Insérer les données de test avec `sql/donnees.sql` (optionnel).
3. Configurer `app/config.php` avec vos identifiants MySQL (par défaut : root, mot de passe vide).
4. S'assurer que le dossier `public/assets/uploads/` existe et est accessible en écriture (permissions 755 ou 777 sous Linux).
5. Accéder au site via `http://localhost/bibliotheque/public/`.

### 3.3 Configuration avancée

- Si vous souhaitez modifier la taille maximale des fichiers uploadés, ajustez la constante `$max = 2 * 1024 * 1024;` dans `app/validations.php`.

- Pour changer les formats d'image autorisés, modifiez le tableau `$allowed` dans `app/validations.php`.

---

## 4. Base de données

### 4.1 Modèle relationnel

La base s'appelle `bibliotheque` et contient trois tables.

Table `livres`

| Champ                          | Type         | Attributs                   | Description                 |
|--------------------------------|--------------|-----------------------------|-----------------------------|
| <code>id</code>                | INT          | AUTO_INCREMENT, PRIMARY KEY | Identifiant unique          |
| <code>titre</code>             | VARCHAR(100) | NOT NULL                    | Titre du livre              |
| <code>auteur</code>            | VARCHAR(100) | NOT NULL                    | Nom de l'auteur             |
| <code>description</code>       | TEXT         | NULL                        | Résumé ou description       |
| <code>maison_edition</code>    | VARCHAR(100) | NULL                        | Éditeur                     |
| <code>nombre_exemplaire</code> | INT          | DEFAULT 1                   | Quantité disponible         |
| <code>image_path</code>        | VARCHAR(255) | NULL                        | Chemin relatif vers l'image |

Table `lecteurs`

| Champ                      | Type         | Attributs                   | Description                   |
|----------------------------|--------------|-----------------------------|-------------------------------|
| <code>id</code>            | INT          | AUTO_INCREMENT, PRIMARY KEY | Identifiant unique            |
| <code>nom</code>           | VARCHAR(100) | NOT NULL                    | Nom de famille                |
| <code>prenom</code>        | VARCHAR(100) | NOT NULL                    | Prénom                        |
| <code>email</code>         | VARCHAR(100) | UNIQUE, NOT NULL            | Adresse email pour connexion  |
| <code>password_hash</code> | VARCHAR(255) | NOT NULL                    | Hash du mot de passe (bcrypt) |

Table `liste_lecture`

| Champ                   | Type      | Attributs                                                               | Description          |
|-------------------------|-----------|-------------------------------------------------------------------------|----------------------|
| <code>id_livre</code>   | INT       | PRIMARY KEY, FOREIGN KEY ( <code>livres.id</code> ) ON DELETE CASCADE   | Référence au livre   |
| <code>id_lecteur</code> | INT       | PRIMARY KEY, FOREIGN KEY ( <code>lecteurs.id</code> ) ON DELETE CASCADE | Référence au lecteur |
| <code>date_ajout</code> | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP                                               | Date d'ajout à la    |

|  |  |  |       |
|--|--|--|-------|
|  |  |  | liste |
|--|--|--|-------|

## 4.2 Requêtes préparées – sécurité

Toutes les interactions avec la base utilisent PDO avec requêtes préparées pour prévenir les injections SQL. Exemple :

```
$stmt = $pdo->prepare('SELECT * FROM livres WHERE id = :id');
$stmt->execute([':id' => $id]);
```

## 5. Fonctionnalités détaillées

### 5.1 Authentification

- Inscription (`register.php`) :
  - o Validation : nom (2-100), prénom (2-100), email valide et unique, mot de passe  $\geq 6$  caractères, confirmation.
  - o Hash du mot de passe avec `password_hash()`.
  - o Redirection vers `login.php` après succès avec message flash.
- Connexion (`login.php`) :
  - o Vérification de l'email et comparaison du hash avec `password_verify()`.
  - o Création de la session (`$_SESSION['lecteur_id']`, `$_SESSION['lecteur_prenom']`).
  - o Redirection vers `index.php`.
- Déconnexion (`logout.php`) :
  - o Destruction de la session et redirection.
- Protection : Les pages `add_book.php`, `edit_book.php`, `delete_book.php` et `wishlist.php` appellent `require_auth()` pour forcer la connexion.

### 5.2 Gestion des livres (CRUD)

| Opération     | Fichier                                           | Description                                             |
|---------------|---------------------------------------------------|---------------------------------------------------------|
| Créer         | <code>add_book.php</code>                         | Formulaire POST → validation → upload image → INSERT.   |
| Lire          | <code>index.php</code> , <code>details.php</code> | Affichage sous forme de cartes ou vue détaillée.        |
| Mettre à jour | <code>edit_book.php</code>                        | Pré-remplissage, validation, upload éventuel, UPDATE.   |
| Supprimer     | <code>delete_book.php</code>                      | Confirmation en POST, suppression de l'image et DELETE. |

### 5.3 Recherche

- `results.php` reçoit le paramètre `q` (GET) et utilise une requête `LIKE` sur `titre` et `auteur`.

- La fonction `rechercherLivres()` retourne un tableau de résultats.

## 5.4 Liste de lecture (wishlist)

- L'utilisateur connecté peut ajouter un livre depuis `details.php` (POST).
- La liste est affichée dans `wishlist.php` ; chaque entrée propose un bouton de retrait (`remove`) avec confirmation JS.
- Les relations sont gérées par les clés étrangères et `ON DELETE CASCADE`.

## 5.5 Upload d'images

- Validation : type MIME réel (`mime_content_type`), taille  $\leq 2$  Mo, extensions autorisées.
- Renommage : préfixe `img_` + date/heure + 4 octets aléatoires en hexadécimal + extension originale.
- Stockage : dans `public/assets/uploads/`.
- Suppression : `delete_image()` supprime le fichier physique lors de la modification ou suppression du livre.

---

# 6. Sécurité et bonnes pratiques

## 6.1 Mesures de sécurité implémentées

- Requêtes préparées : Toutes les requêtes SQL utilisent PDO avec paramètres nommés.
- Échappement HTML : Fonction `e()` basée sur `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` appliquée à chaque affichage de données utilisateur ou base.
- Validation des formulaires : Côté serveur (`validate_livre`) et côté client (JS) pour une meilleure UX.
- Hash des mots de passe : Utilisation de `password_hash()` (bcrypt).
- Sessions sécurisées : Le cookie de session est géré par PHP (configuration par défaut).
- CSRF : Non implémenté dans ce projet (peut être ajouté en bonus).
- Upload : Contrôle strict des types MIME, renommage, taille limitée.

## 6.2 Bonnes pratiques de codage

- Modularité : Le code métier est découpé en fichiers séparés (helpers, validations, fonctions).
  - DRY : Éviter la duplication (ex: fonction `e()` pour tout l'affichage).
  - Commentaires : Les fonctions et sections principales sont commentées en français.
  - Responsive : Utilisation de CSS Grid et Flexbox, media queries.
  - Écoconception : Pas de bibliothèques externes lourdes ; images optimisées ; requêtes limitées.
-

## 7. Interface utilisateur – Guide rapide

### 7.1 Accueil ([index.php](#))

- Barre de navigation (connexion, wishlist, ajout livre).
- Champ de recherche.
- Liste des 10 derniers livres (cartes avec image, titre, auteur, nombre d'exemplaires, lien "Voir détails").

### 7.2 Détails d'un livre ([details.php](#))

- Image à gauche (ou placeholder), titre, auteur, maison d'édition, exemplaires, description.
- Boutons : Ajouter à la wishlist (connecté), Modifier, Supprimer.

### 7.3 Gestion de la wishlist ([wishlist.php](#))

- Liste des livres sélectionnés avec date d'ajout.
- Bouton "Retirer" pour chaque livre.

### 7.4 Formulaire

- Tous les formulaires possèdent des champs obligatoires marqués par \*.
  - Des messages d'erreur précis s'affichent en cas de saisie invalide.
  - En cas de succès, un message flash (vert) apparaît.
- 

## 8. Tests et validation

### 8.1 Tests manuels effectués

- Création d'un compte utilisateur.
- Connexion / déconnexion.
- Ajout de livres avec et sans image.
- Modification de livre (changer image, conserver image).
- Suppression de livre avec confirmation.
- Recherche par mot-clé (titre ou auteur).
- Ajout/retrait de la wishlist.
- Vérification de l'intégrité des données après suppression d'un livre lié à une wishlist.

### 8.2 Problèmes connus et solutions

- Image non affichée : Vérifier les permissions du dossier [uploads/](#).
- Erreur de connexion DB : Vérifier les identifiants dans [config.php](#).

- Erreur d'upload : Taille limite dépassée ou format non autorisé.
- 

## 9. Évolutions possibles

- Pagination des résultats de recherche et de l'accueil.
  - Gestion des emprunts avec dates de retour.
  - Système de notation / commentaires sur les livres.
  - Administration avec rôles (bibliothécaire, lecteur).
  - Export du catalogue en CSV.
  - API REST pour une éventuelle application mobile.
- 

## 10. Annexes

### 10.1 Exemple de message flash

```
set_flash('ok', 'Livre ajouté avec succès.');
```

// Dans la vue :

```
if ($flash = get_flash()) {  
    echo '<div class="alert ' . $flash['type'] . '"> ' . $flash['message'] . '</div>';  
}
```

### 10.2 Fonction d'échappement

```
function e($v) {  
    return htmlspecialchars((string)$v, ENT_QUOTES, 'UTF-8');  
}
```

### 10.3 Exemple de requête préparée

```
$stmt = $pdo->prepare('UPDATE livres SET titre = :titre WHERE id = :id');  
$stmt->execute([':titre' => $titre, ':id' => $id]);
```

---

Documentation rédigée par : Giovanni MARC Date : Juillet 2026 Version : 1.0 Licence : Libre d'utilisation pour un usage pédagogique.